Clinic Appointment System

Authors: R2420439, R2420437 (group 5) **Language:** Python 3.10+ (standard library only, nothing to install)

P = 5 (our group number, one constant in clinic/config.py): doctor fee 40, nurse fee 12, pharmacist fee 5.

P mod 4 = 1, so our twist is **Emergency**.

How to run (VS Code)

1. Install the **Python** extension if VS Code asks, and pick an interpreter (Python 3.10 or newer) via **Ctrl+Shift+P** > **Python: Select Interpreter**.
2. **Run the demo:** open main.py and press the Run button (or **F5**, choose "Run demo (main.py)"). Or in the VS Code terminal (**Ctrl+`**):

   ```
   python main.py
   ```

   (On Mac/Linux use python3 main.py if python is not found.)
3. **Run the tests:** in the terminal:

   ```
   python -m unittest discover -v -s tests -t .
   ```

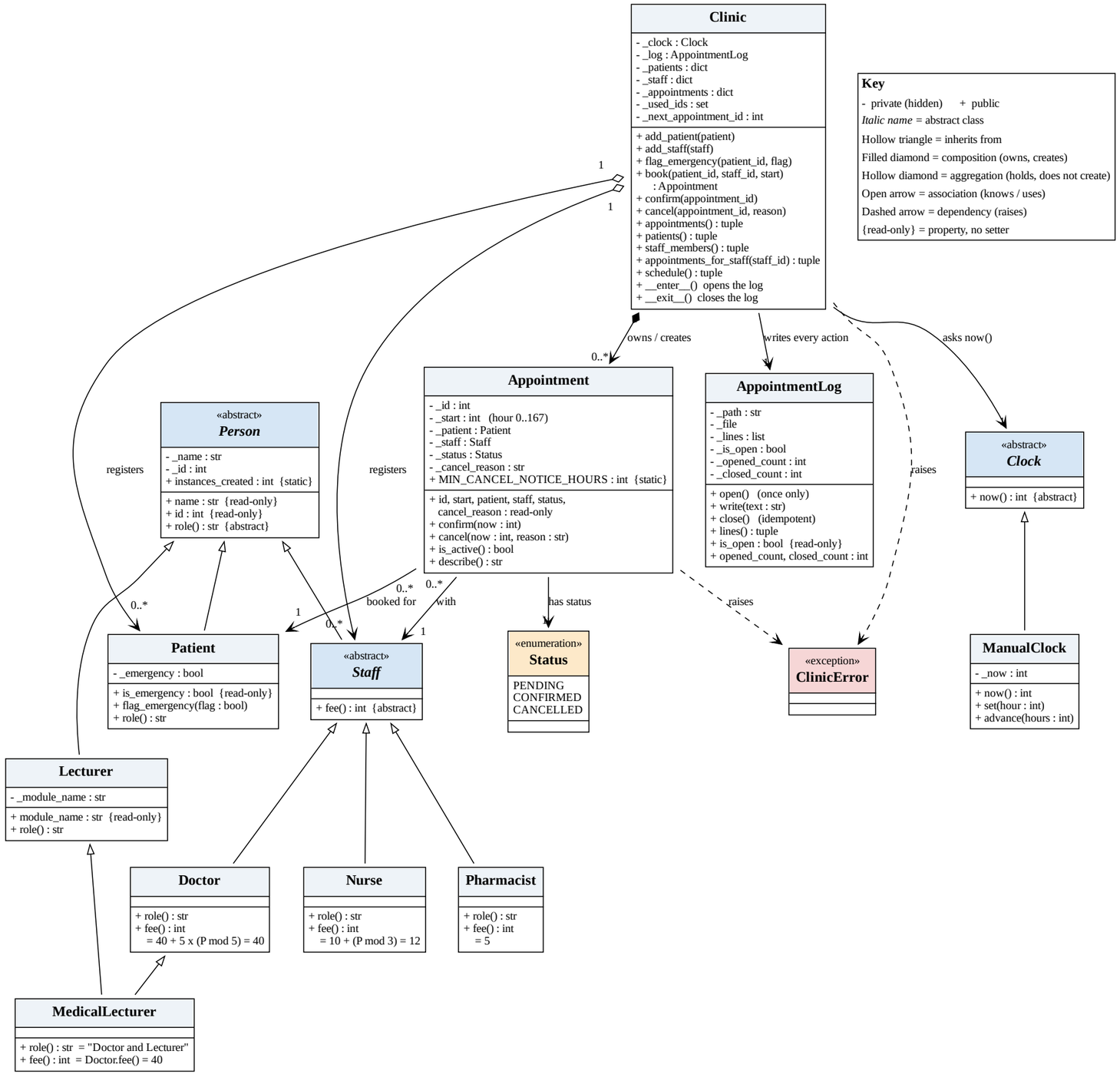
   (you should see 22 tests and OK), or press F5 and choose "Run all tests", or use the Testing (beaker) icon in the sidebar.
4. **Optional dashboard (browser):** in the terminal run python dashboard.py (or F5 > "Run dashboard"). It opens http://127.0.0.1:8000 where you can book, confirm, cancel, flag emergencies, move the clock forward and watch the live log. Press **Ctrl+C** in the terminal to stop it; the log is closed properly ([log] closed). If port 8000 is busy: python dashboard.py --port 8001.
5. The demo writes a log file to logs/appointments.log (the dashboard writes logs/dashboard.log). Always run from the project folder that contains main.py (VS Code does this when you open that folder).

Project layout

```
clinic_appointment_system/
  main.py                      5-minute demo
  clinic/                      the system (a Python package)
    people.py                  Person, Patient, Staff, Doctor, Nurse, Pharmacist, Lecturer, MedicalLecturer
    appointment.py             Appointment, Status, legal transitions, cancellation rules
    clinic.py                  Clinic: booking rules, Emergency twist, logging of every action
    log.py                     AppointmentLog (open once, close once, survives its own failures)
    clock.py                   Clock (abstract), ManualClock, format_hour
    validation.py, errors.py, config.py
  dashboard.py                 optional browser dashboard (no rules inside, only calls Clinic)
  tests/                       22 unit tests (test_clinic.py, test_dashboard.py)
  docs/                        REQUIREMENTS_CHECKLIST.md, CHANGE_REQUEST_PLAYBOOK.md, demo_output.txt
  .vscode/                     one-click run and test settings
```

1. Class diagram

Figure 1: Clinic Appointment System, class diagram (group 5, P = 5)



2. Encapsulation and the appointment state machine

- **Hidden fields:** every field is _private (Python's convention, not enforced by the language) and exposed only through read-only @property getters. There are no setters except deliberate operations (confirm, cancel, flag_emergency). In Java these would be private final.
- **No leaked lists:** Clinic.appointments(), staff_members(), patients() and AppointmentLog.lines() return **tuples built as copies**, so callers cannot change Clinic's internal dicts or lists. (Java: List.copyOf.)
- **Status changes** only through confirm(now) and cancel(now, reason). Legal transitions: PENDING -> CONFIRMED, PENDING -> CANCELLED, CONFIRMED -> CANCELLED. CANCELLED has no outgoing transitions (_LEGAL table in appointment.py).
- **Rules enforced in** Appointment.cancel**:** a cancelled appointment never returns (empty transition set); now >= start means "already started" and blocks cancel and confirm; a CONFIRMED appointment with start - now < 2 fails with "too late" (exactly 2 hours is allowed).
- **Constructors:** Person.__init__ calls clean_text (rejects empty / whitespace names) and positive_id (rejects ids <= 0) and raises ValueError. These checks use strip() and operator.index(), so there is no type test.
- **Invariants** are written as comments above Appointment and Clinic.

3. Polymorphism without type checks

Staff.fee() and Person.role() are abstract. The loop is just for m in staff: print(m.role(), m.fee()). main.py defines a new LabTechnician(Staff) and it works in the loop and in Clinic.book with no edits. A test scans the source for isinstance/issubclass.

4. Logging guarantee

Mechanism: the context-manager protocol (with**).** Clinic.__enter__ opens the log and Clinic.__exit__ closes it. __exit__ always runs, even when the body raises, and AppointmentLog.close() is idempotent, so the log is closed exactly once. Every public Clinic action runs inside _action(label), which logs FAILED <action>: <reason> and re-raises. The demo shows [log] opened, [log] closed and [log] FAILED cancel #3: too late, and a crash mid-session still prints [log] closed.

5. Diamond: Lecturer and MedicalLecturer

MedicalLecturer(Doctor, Lecturer) has MRO MedicalLecturer, Doctor, Staff, Lecturer, Person. Every __init__ takes keyword arguments and calls super().__init__(**kwargs), so Person.__init__ runs once. Person.instances_created shows +1 in the demo and in a test. role() returns "Doctor and Lecturer". fee() **returns the Doctor fee (40)**: the fee is for the clinical service, and teaching is not billed to patients. It is written explicitly (Doctor.fee(self)) so it does not depend on MRO order.

Java version: Java has no multiple class inheritance, so Lecturer becomes an **interface** (getModuleName()). MedicalLecturer extends Doctor implements Lecturer and keeps the module name in its own field, because interfaces cannot hold state. There is exactly one Person superclass chain, so Person is constructed once. A stand-alone lecturer then needs a small class, e.g. PlainLecturer extends Person implements Lecturer.

6. Rules between objects and the Emergency twist

- One person-id namespace (patients and staff) is checked in Clinic._register. Appointment ids are assigned by the Clinic and never reused.
- No staff member and no patient may have two non-cancelled appointments at the same hour. A cancelled appointment frees its slot. A booking must start in the future.
- A broken rule raises ClinicError, changes nothing, and is logged as FAILED book ...: reason.
- **Emergency:** flag_emergency(patient_id). An emergency patient booking a staff slot taken by a **PENDING** appointment cancels it with reason "bumped" (logged) and books the slot. If the slot holds a CONFIRMED appointment, the booking fails. All checks run **before** anything is cancelled, so a failed booking never leaves a

half-done bump. The patient's own clash can never be bumped.

7. Design decision log

D1. Where does the 2-hour rule live? In Appointment.cancel, as MIN_CANCEL_NOTICE_HOURS. The appointment owns its status and start, so the rule is part of its invariant and cannot be bypassed by anyone holding an appointment. Considered: *Clinic* (any other caller of Appointment.cancel would skip it); *Staff* (useful if notice differed per role, but the rule is currently the same for everyone); a *policy object* (most flexible if clinics need different rules, at the cost of an extra class and wiring). Trade-off: simplicity and safety now, versus flexibility later. The clock is passed in as now, so the rule stays pure.

D2. Why is the clock testable? Clock is an abstract now(), injected into Clinic. Tests use ManualClock and set the hour exactly (19, 20, exactly 2 h before). With the system time we could not test the boundaries deterministically, could not test "already started" without waiting, could not map real time onto the 0-168 hour model, and tests would be flaky and slow.

D3. Adding Pharmacist: nothing besides the new class (role() and fee()); the demo's LabTechnician proves the same. Nothing leaked into Clinic or the loop. The only other edit is where the demo creates one.

D4. MedicalLecturer.fee() returns 40 (Doctor fee), see section 5. **Cooperative** super() **and the MRO** stop Person being created twice. Without it, if Doctor and Lecturer each called Person.__init__(...) directly, Person would run twice: the counter would be 2, and a second run would re-initialise (overwrite) the name and id; in a real system it could register or allocate an id twice.

D5. If writing to or closing the log fails: a log problem must not corrupt or block clinic work and must never hide the original error. AppointmentLog catches OSError/ValueError, prints a one-time [log] WARNING, keeps all lines in memory (lines()), and continues. A failed close still marks the log closed and never raises. Trade-off: the file may miss entries (the warning says so); an audited clinic could instead choose to stop accepting bookings.

D6. Twist (P mod 4 = 1, Emergency). *Why can't a confirmed appointment be bumped?* A CONFIRMED appointment is a commitment that both the patient and the staff member have accepted, and the patient may already have arranged transport, work or childcare around it. Bumping it would break a promise and make confirming meaningless. Only tentative (PENDING) appointments can be displaced, and the bump is logged with reason "bumped". Also, the 2-hour rule already protects CONFIRMED appointments close to their start, so emergencies never get a back door around it. *(Sub-questions 0, 2 and 3 of D6 belong to other values of P mod 4 and do not apply to our twist.)*

Optional: dashboard (dashboard.py)

A small local web page built with Python's standard library. It is **not** part of the required design; it is a thin view on top of it.

- **Separation of concerns:** the dashboard has no rules. Each button becomes one call on Clinic (book, confirm, cancel, flag_emergency) and the page draws what Clinic.appointments(), Clinic.schedule() and AppointmentLog.lines() return. A broken rule shows up as a "Refused: ..." message and a red FAILED line in the log, exactly as in the console demo.
- **Lifecycle:** the server runs inside with Clinic(...), so stopping it with Ctrl+C still closes the log once.
- **Safe by default:** it listens on 127.0.0.1 only (your own computer).